De 0 a Python en 45'

Una breve introducción a Python

Autor:
- Daniel F Moisset - dmoisset@except.com.ar - Except

Modificaciones:
- Natalia B Bidart - nataliabidart@gmail.com

(cc) BY-NC-SA

Resumen de la charla

- Introducción
- Conceptos generales (con código)
- Código
- Más código
- Ejemplos
- Conclusión (python rulez)

(cc) BY-NC-SA

Introducción

- Python es un lenguaje creado en 1991 por Guido van Rossum
- Software Libre
- VHLL, interpretado, OO, dinámico
- Versión actual: 2.5 (2.4 se usa mucho)
- “Viene con las pilas incluidas”

python™ http://python.org

(cc) BY-NC-SA

Usando python

- Modo interactivo

```
python
Python 2.4.2 (#2, Sep 30 2005, 21:19:01)
[GCC 4.0.2 20050808 (prerelease) (Ubuntu
  4.0.1-4ubuntu8)] on linux2
Type "help", "copyright", "credits" or "license" for
  more information.
>>> a = 3+8
>>> print a
11
>>> 6 * 9
54
```

- Modo scripting

```
python programa.py
```

(cc) BY-NC-SA

Tipos básicos y valores

- Enteros (`int`/`long`): `0`, `1`, `42`, `-7`, `1234567890123456789099 9`
- Reales (`float`): `3.14`, `.5`, `2.1e25`
- Complejos (`complex`): `3.2j`, `(-1+2.7j)`
- Booleanos (`bool`): `True`, `False`
- `NoneType`: `None`

(cc) BY-NC-SA

Operadores básicos

- Aritmética (+, -, *, /, %, **)

```
>>> 6+5
11
>>> 6+2.5
8.5
```

```
>>> 16/3
5
>>> 16%3
1
```

```
>>> 16.0/3
5.333333333
>>> 7**17
232630513987207L
```

- Comparación (==, !=, <, >, <=, >=)

```
>>> 3==7
False
>>> 3!=7
True
```

```
>>> 3<=3
True
>>> 1<5<=9
True
```

```
>>> 1<8<3
False
>>> None==47
False
```

- Lógica (and, or, not)

```
>>> (3==7) or False
False
>>> not (1==0)
True
```

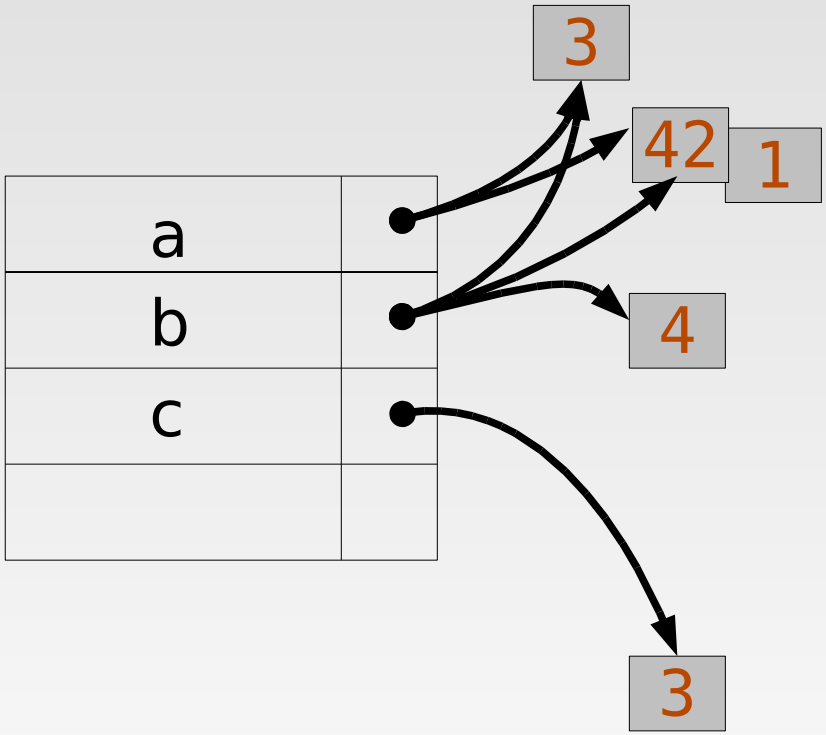
```
>>> True and False
False
>>> not None
True
```

(cc) BY-NC-SA

Namespaces

- Nombres: a-z, A-Z, 0-9, _

```
# Creación, Asociación
a=3
# Asociación
b=a
# Otra vez...
c=3
# Disociación
del a
# Reasociación
b=b+1
# Multiasignación
a=b=42
```



(cc) BY-NC-SA

Secuencias

- Cadenas (`string`/`unicode`)

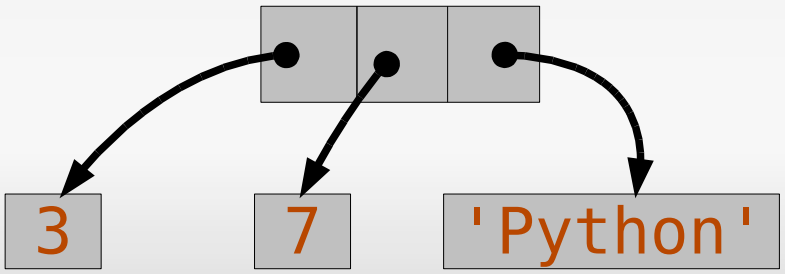
```
'Richieri', "O'Higgins", u'San Martín'
"""Aquí me pongo a cantar
al compás de la vigüela..."""
```

- Listas (`list`)

```
>>> [3,2+5,"Python"]
[3, 7, 'Python']
>>> []
[]
```

- Tuplas (`tuple`)

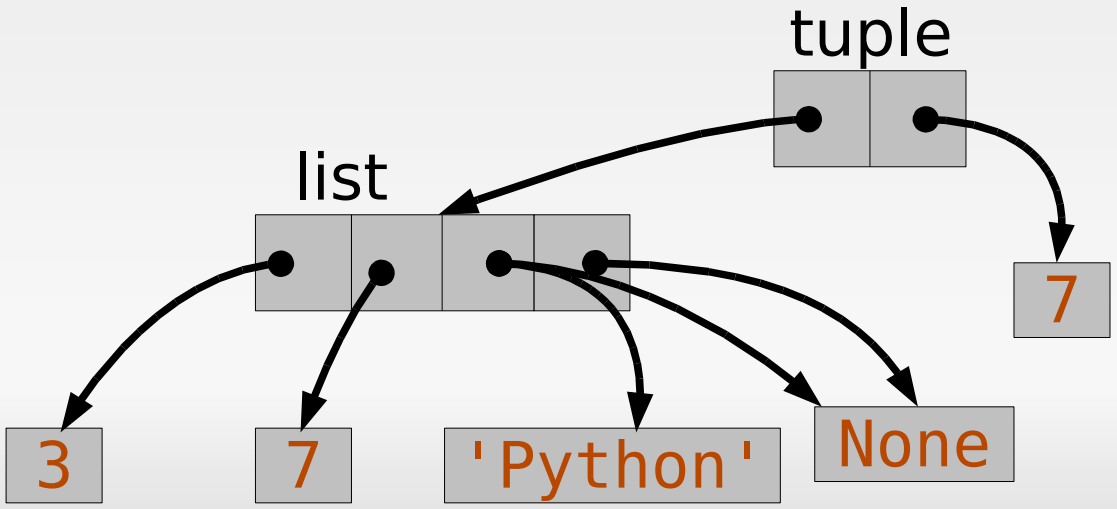
```
>>> (3,2+5,"Python")
(3, 7, 'Python')
>>> (1,)
(1,)
>>> ()
()
```



(cc) BY-NC-SA

Mutables e Inmutables

- Sólo los objetos “mutables” pueden alterarse entre su nacimiento y muerte
- Mutables: listas, diccionarios, tipos de datos “caseros” (clases)
- Inmutables: el resto de las cosas que vimos.



(cc) BY-NC-SA

Operaciones sobre secuencias

- Acceso, slicing

```
>>> lista=[1,2,3,[]]
>>> cadena='Python!'
>>> lista[2]
3
>>> cadena[-1]
'!'
>>> cadena[3:5]
'ho'
>>> lista[1:]
[2, 3, []]
```

-7	P	0
-6	y	1
-5	t	2
-4	h	3
-3	o	4
-2	n	5
-1	!	6

- Concatenación

```
>>> 'ja'*3
'jajaja'
>>> (1,2)+(3,4)
(1, 2, 3, 4)
```

- Comparación

```
>>> 'Adam'<'Alan'
True
>>> (1,2,3)>(1,7)
False
```

- Asignación*

```
>>> lista[0]=cadena
>>> lista
['Python!', 2, 3, []]
>>> lista[1:3]='xy'
>>> lista
['Python!','x', 'y', []]
>>> lista[1:]=[]
>>> lista
['Python!']
```

(cc) BY-NC-SA

Operaciones en secuencias

- Pertenencia

```
>>> 2 in [1,2,3,[]]
True
>>> 'te' in 'tomate'
True
>>> [2,3] in [1,2,3,[]]
False
```

- Conversión

```
>>> list ('casa')
['c', 'a', 's', 'a']
>>> tuple([1,2,5])
(1, 2, 5)
>>> str([1,2])
'[1, 2]'
```

- Borrado*

```
>>> lista=[5,7,1,4,3.2]
>>> del lista[0]
>>> lista
[7, 1, 4, 3.2]
>>> del lista[2:]
>>> lista
[7, 1]
```

(cc) BY-NC-SA

Builtins

```
>>> len
<built-in function len>
>>> len('Python!') + len([1,2,3])
10
>>> f=len
>>> f('Python!') + f([1,2,3])
10
>>> range(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> range (8, 13)
[8, 9, 10, 11, 12]
>>> range(8,25,5)
[8, 13, 18, 23]
>>> l =  [-1, 3, 5, 8, 42, 7]
>>> min(l),  max(l), sum(l)
(-1, 42, 64)
>>> sorted(l)
[-1, 3, 5, 7, 8, 42]
```

(cc) BY-NC-SA

More Builtins

```
>>> abs (-7.5), round (7.5)
(7.5, 8.0)
>>> x=input('Ingrese valor: ')
Ingrese valor: 3+7
>>> x
10
>>> y=raw_input('Ingrese valor: ')
Ingrese valor: 3+7
>>> y
'3+7'
>>> f=file('datos.txt')
>>> help(input)
```

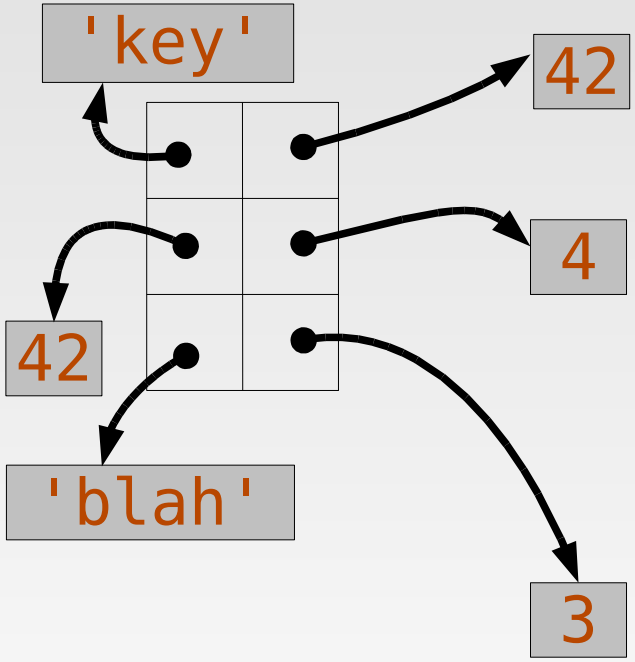
```
Help on built-in function input in module __builtin__:

input(...)
    input([prompt]) -> value

Equivalent to eval(raw_input(prompt)).
```

(cc) BY-NC-SA

Diccionarios

- Asociaciones Clave → Valor
- Acceso rápido

```
>>> d={'key':42, 42:4, 'blah':3}
>>> d['key']
42
>>> len(d)
3
>>> d['vacia']=[]
>>> del d[42]
>>> 42 in d
False
>>> list(d)
['blah', 'key', 'vacia']
>>> d.values()
[3, 42, []]
```



(cc) BY-NC-SA

Manejando objetos

- *o.nombre* accede al namespace dentro de *o*

```
>>> d.values
<built-in method values of dict object at 0xb7d874c>
>>> (3.5+2j).imag
2.0
>>> l=[1,3,5]
>>> l.append(9)
>>> l
[1, 3, 5, 9]
```

- `dir`(*o*) lista los elementos del namespace de *o*

```
>>> dir ([1,3,5])
...'append', 'count', 'extend', 'index', 'insert', 'pop',
'remove', 'reverse', 'sort']
```

(cc) BY-NC-SA

Algunas instrucciones

- Output:

```
print 'Hello'     print 'Hello',n      print x,y,
```

- Condicional:

```
if not n in d:
   print 'No está'
```

```
if len(l)>1:
   print 'un archivo'
else:
   print len(l),'archivos'
```

```
if fecha>hoy:
   color = 'red'
elif fecha==hoy:
   color = 'yellow'
else:
   color = 'green'
```

(cc) BY-NC-SA

Otras instrucciones

- Ciclos:

```
for renglon in file('texto.txt').readlines():
    if 'python' in renglon:
        print renglon,

while n != 1:
    if n % 2==0:
        n = n/2
    else:
        n = 3*n + 1
```

- `break`, `continue`
- `try/finally`, `try/except`, `assert`, `raise`
- `pass`

(cc) BY-NC-SA

Módulos

- Los módulos son objetos compartidos
- Hay módulos estándares, bibliotecas, de usuario

```
>>> import os, re
>>> os.system ('cal')
    August 2006
Su Mo Tu We Th Fr Sa
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31

0
>>> from os import system, popen
```

(cc) BY-NC-SA

Funciones

- Con **def**, devuelven valores con **return**

```
def menu(mensaje, opciones):
    """muestra opciones, mensaje y pide un valor"""
    print ('MENU:')
    for opcion in opciones:
        print opcion, opciones[opcion]
    return raw_input(mensaje)
```

- Parámetros opcionales

```
def sumar_iva(monto, iva=21):
    total = monto * (100.0+iva)/100
    return total
print sumar_iva(311.75), sumar_iva (67, 10.5)
```

- Parámetros variables

```
def promedio (*l):
    return sum(l)/len(l)
```

(cc) BY-NC-SA

Clases

- Tipos de objetos y sus operaciones

```
class Actor:
    """Actor con filmografía"""
    def __init__(self, nombre):
        self.nombre = nombre
        self.filmografia = []
    def actuar (self, pelicula)
        self.filmografia.append(pelicula)
    def actuoEn (self, pelicula)
        return (pelicula in self.fimografia)
    def actuoCon (self, actor):
        for pelicula in self.filmografia:
            if actor.actuoEn(pelicula):
                return True
        return False
a = Actor('Hugo Weaving')
a.actuar ('Babe')
a.actuar ('Matrix')
if a.actuoCon (k): ...
```

(cc) BY-NC-SA

Conclusiones

- Es fácil!
- Se escribe y se lee rápido
- Es práctico (y viene con las pilas puestas)
- Es poderoso
- Es Software Libre
- Tiene una gran comunidad:

PyAr http://python.com.ar/

(cc) BY-NC-SA

Preguntas?

Autor: Daniel F Moisset
dmoisset@except.com.ar

Modificaciones: Natalia B Bidart
nataliabidart@gmail.com

(cc) BY-NC-SA

Que cosas puedo hacer?

- Scripts "para salir del paso"
- Aplicaciones de BDD
- Cliente/Servidor
- Servicios Web, Aplicaciones Web
- Software de Escritorio
- etc...

(cc) BY-NC-SA

En que plataformas corre

- GNU/Linux
- Windows 9x, NT, 2000, XP, ...
- Mac OS Classic, Mac OS X
- *BSD, Solaris, varios Unices
- Embedded (Palm)
- Nokia 6000 series
- y más....

(cc) BY-NC-SA